VEDHYAR

UNDERSTAND WHAT YOU BUILD.

For people building with AI

Own the next change.

Understand the logic. Make your edit.
Check it against your original idea.

Understand → Change → Verify



Who is Vadhyar for?

People building with AI who want to own their next change.

Non-developers building with AI

“It works, but I can’t explain it.”

Understand the feature. Make one guided edit.

Students & beginners

“Help me understand the concepts in my project.”

Predict, investigate, apply. Record the help used.

Hobbyists & founders

“I need to change this without breaking it.”

Inspect the affected logic. Check old and new cases.

Developers

Find the relevant logic quickly.

Mentors & teachers

Review reasoning, hints and actual results.

Primary focus: a beginner’s first change to AI-built code.

A working app can still leave you stuck.

“AI built it. Where do I change it?”

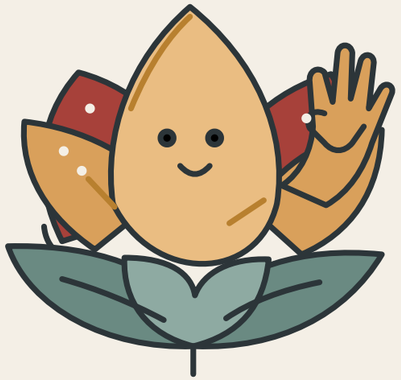
A beginner’s first change requires finding the decision and understanding its effect.

66%

Nearly-correct outputs were a frustration.

Developer respondents. Not a beginner study.

One feature becomes a guided learning path.



Follow the logic

Your requirement.
Linked project code.

Predict behavior

Commit your reasoning.
Run the case.

Make your edit

Edit in VS Code.
Inspect real checks.

Confirm | Understand | Predict | Observe | Change | Verify

The happy path hides an ownership bug.

Original requirement

Only the owning client may approve a milestone.



Actor	Rule says	Code does
Owning client	Allow	**Allow**
Different client	Deny	**Allow**
Freelancer	Deny	**Deny**

The code allowed it.
The rule says it shouldn’t.

Watch a learner take the next change.



Switch to VS Code

Prediction → hint → human edit → checks

Show the current code, the source link, the native diff, and the actual result.

01 Predict and observe

02 Investigate and edit

03 Run three cases

90-second live demonstration

Assistance is visible. The learner owns the edit.

Prepared local workspace · real API request · learner edit · executed checks

OpenAI coaches. Executed checks establish behavior.



Project evidence

Requirement + source
+ Graphify relationships





OpenAI API

Explain the decision.
Offer a cited hint.





The learner

Inspect the source.
Make a saved edit.



Vitest executes the cases. Results stay tied to the code version.

A human edit changed the observed result.

Recorded installed acceptance

Live explanation + conceptual hint.
Saved developer edit + native diff.



Predicted first

Before the case ran



Developer edit

Test authorship disclosed



Transfer matched

405 ms · none skipped

These cases establish behavior. They do not measure mastery.

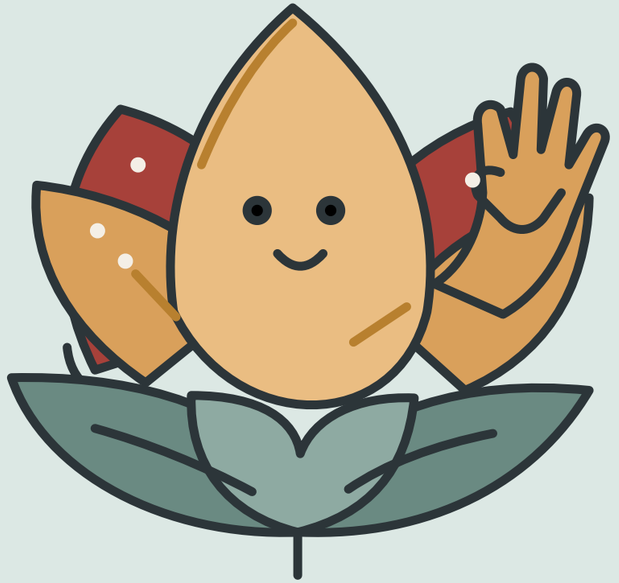
Source: S13 installed acceptance · prepared developer rehearsal, not mastery

Your project. Your next decision.

Build with AI. Understand what you build.

Try one feature.
Explain it. Change it. Check it.

github.com/sabari-nath17/vadhyar →



Vadhyar

Study observable strengths, not a winning formula.

1. **A specific difficult moment**

   Second Voice / veTriage → first feature change

2. **An inspectable mechanism**

   Mechanica / Echo Canvas → source-linked decisions

3. **Measured output beside guidance**

   Dấu / Sentinel → executed checks beside hints

4. **Human control at the boundary**

   AirBridge / Pulse → reviewed context and human edits

Personal problems became usable, bounded workflows.

Winner / category	Documented strength	Lesson for Vadhyar*
Second Voice 1st · Apps for Your Life	Few sentence choices; confirmation before speech.	Few clear next actions; learner keeps control.
AirBridge 2nd · Apps for Your Life	Working audio routing; measured calibration.	Run real behavior; disclose system limits.
veTriage 1st · Work & Productivity	Clinic-specific handoffs; reported routine pilot.	Show a specific workflow; seek actual user feedback.
Pulse 2nd · Work & Productivity	Speech evidence feeds auditable deterministic state.	Keep evidence categories and execution distinct.

*Design interpretation · builder claims were not independently reproduced

Hidden systems became tangible and testable.

Winner / category	Documented strength	Lesson for Vadhyar*
Echo Canvas 1st · Developer Tools	Hear changes while inspecting acoustic paths.	Make one code decision and its effect visible.
Sentinel 2nd · Developer Tools	Cited review plus real isolated probes.	Validate references; execute the selected case.
Mechanica 1st · Education	Operable reconstructions; source / inference labels.	Follow the mechanism; identify uncertainty.
Dấu 2nd · Education	Pitch beside a reference; DSP verdict, AI coaching.	Put required and actual behavior beside the hint.

*Design interpretation · no causal ranking claim

Memory stays local. Sharing has a review step.

Workspace memory



Requirement revisions

Source snapshots + hashes

Predictions, hints + check history

VS Code extension storage



Credentials

VS Code SecretStorage



OpenAI requests

Review selected context first.



Execution

Trusted workspace + fixed checks

Implemented features have different evidence levels.

Recorded native

Working demonstrations

Ownership + archived transfer

SeatCheck project explanation

Graphify learning topics

Saved diffs, timeline + resume

Implemented; limits

Check before relying on

Build-plan / prompt operation

Capture bridge connection

CodeTour-compatible export

Latest packaged-build parity

Proposed / deferred

Future work

v2 lesson UI + stickers

OAuth + cloud memory

Whole-project PRD auditing

Alternate solutions + repair

The contribution is the connected handover.

Reuse

Graphify

Source relationships

VS Code + Vitest

Native editing + execution

CodeTour / OpenSpec

Walkthrough / requirement formats

Vadhyar connects

Requirement
→ prediction
→ learner edit
→ observed result

On one evolving project feature.

Reuse modes and notices: docs/reuse-sources.md

Our workflow has specific reusable foundations.

Vadhyar feature	Source / reuse mode	Our implementation
Source relationships Two bounded adapters	Graphify · external CLI	Validated static references; requirement-linked context.
Guided walkthrough Implemented format adapter	CodeTour · safe tour format	Local navigation and export; inert content fields.
Requirement import Implemented local parser	OpenSpec · Markdown convention	Confirm rule + revision; cases stay unverified.
Executed checks Native + dependency	Vitest + native VS Code	Fixed prepared cases, hashes, real outcomes and native diff.

Teaching and design inspirations are named.

Vadhyar choice	Inspiration	Adaptation / boundary
Prediction before edit Teaching approach	PRIMM · predict, run, investigate, modify, make	Own project + requirement; archived task works.
Checkpoints + diff Reference, not copied code	Cline · checkpoints	Native snapshots and diffs; no imported agent runtime.
Hints beside evidence Winner design lessons	Dấu + Sentinel	Coach / runner separation. No winner code imported.
Friendly lesson path Proposed application UI	Duolingo marketing feel; user's design-v2.md	Vadhyar palette + Guide; original assets and artwork.

Track broadly. Teach one decision.

Recorded T1 topic

Positive seat capacity

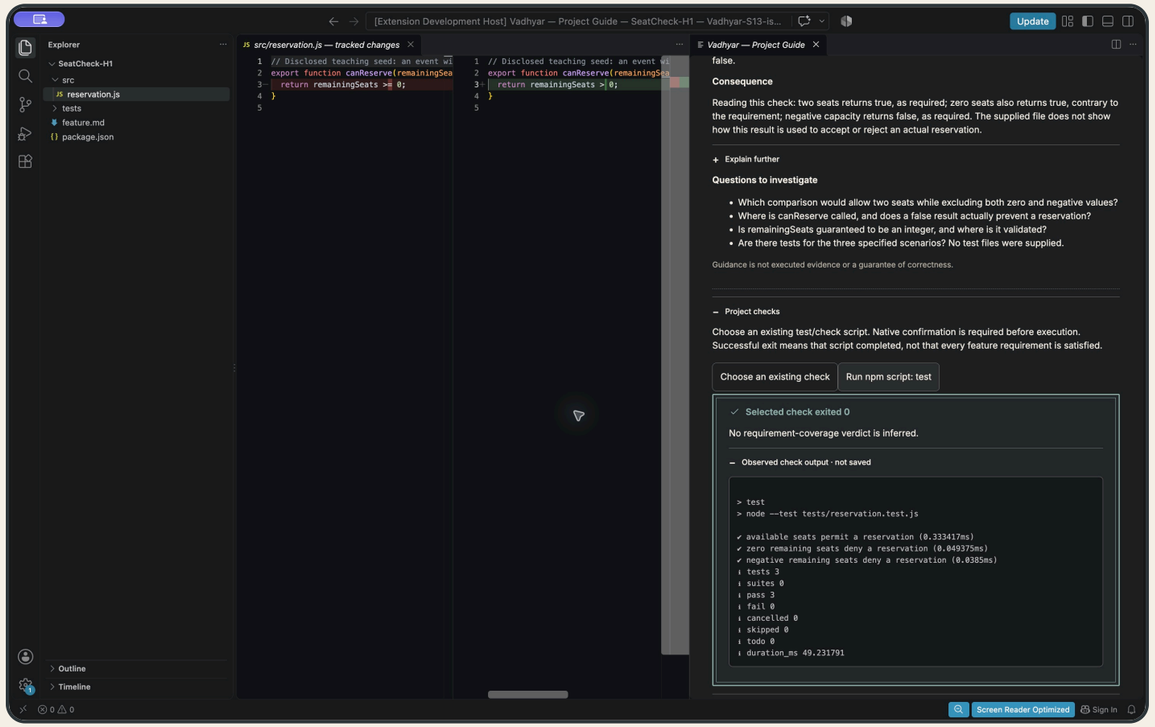
Only positive integer seat counts should be accepted.

```
Number.isInteger(seats)
  && seats > 0
```

Confirmed source: src/reserve.js:2

1. Requirement · explicitly confirmed
2. Decision · exact current condition
3. Reason · model proposal to review
4. References · extracted calls/imports
5. Coverage · 1 of 5 captured files reviewed

SeatCheck demonstrates a second project.



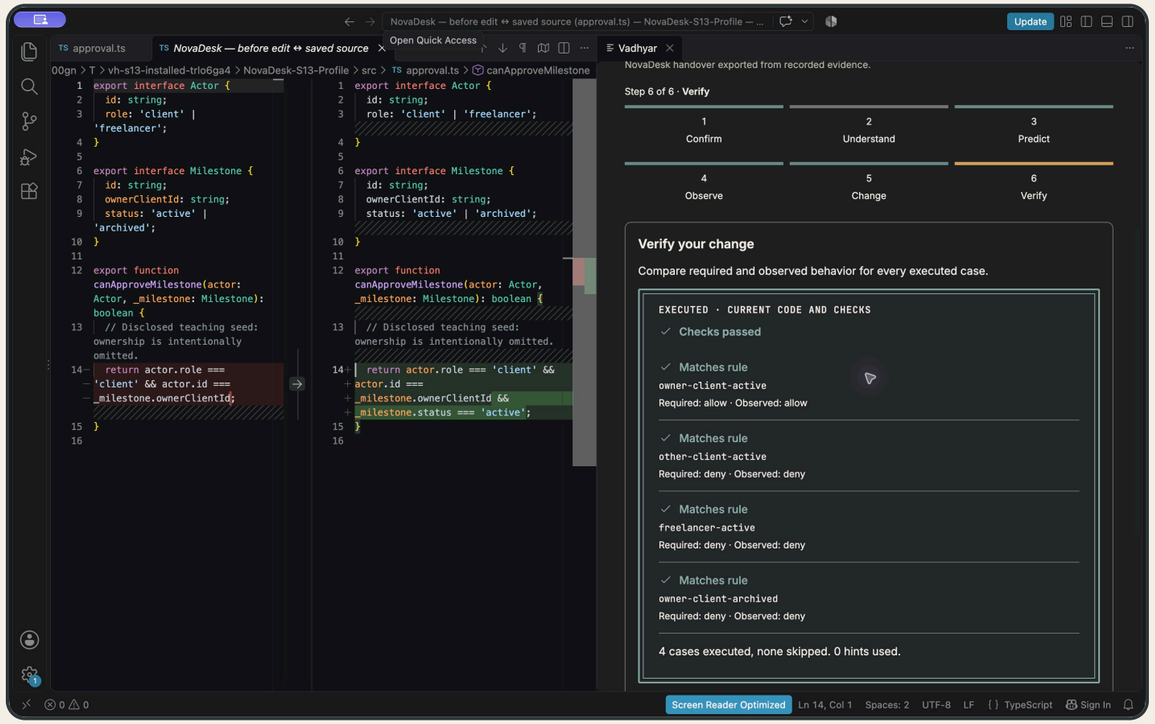
Recorded H1 acceptance

Reserve only an available seat.

Confirm imported intention.
Review live source explanation.
Save an edit + inspect diff.
Run the same selected check.

Seed: 2 pass / 1 fail
After edit: 3 pass / 0 fail

The archived follow-up is implemented.



Recorded installed S13

Archived milestones stay blocked.

Separate rule + prediction.
Actual archived counterexample.
Saved edit + native comparison.
Four fixed cases executed.

4/4 cases matched · no skips
14-row timeline + Markdown export

The current source includes these features.

Intent + context

Track the project

- Add / revise feature intentions
- Import and confirm PRD text
- Saved source snapshots
- Native tracked file diffs
- Relevant-file selection

Guidance

Investigate a decision

- Grounded learning topics
- Source-linked explanations
- Progressive NovaDesk hints
- Reviewed build plan + prompt*
- CodeTour-compatible export*

Evidence + control

Keep the record

- Prediction and actual cases
- Archived follow-up + reflection
- Timeline / Markdown handover
- Local memory + stale guards
- Optional local Snake break*

*Implemented with narrower acceptance · see Build-to-Deck-Audit.md

Know which artifact contains each feature.

Current source

Development checkout

T1 grounded learning topics
Static Graphify project links
Latest privacy / consent fixes
Recorded source tests: 134/134
Package gate pending

Local VSIX · 05:50

Installed candidate

NovaDesk ownership + archive
SeatCheck explanation / check
Native resume recorded
Predates T1 and safety patch
macOS Apple silicon only

Public S7 installer

Earlier distribution

Narrower published scope
Separate from local candidate
No latest-feature promise
Not the current source build
Submission SHA not recorded

Source, compiled bundle and installer differ · exact-artifact rehearsal is required

Consent follows the current code.

1 **Review selected context**

Conservative private-text withholding; the detector remains heuristic.

2 **Confirm the existing script**

The chosen npm script runs; implicit pre/post hooks are disabled.

3 **Recheck after confirmation**

Saved or dirty source drift blocks the approved execution.

4 **Keep failures explicit**

Cancellation, timeout and abnormal exit stay separate from completion.

Learning value must survive an unfamiliar change.

1

Investigate a guided feature

Record prediction, explanation, time and hints.

2

Attempt an unfamiliar next rule

Observe reasoning, completion and regressions.

A passing check measures code behavior.
Transfer needs a separate learner observation.

The sources remain inspectable.

Competition and submissions

Official winners announcement

Published Build Week judging criteria

All eight submission links: slides 11–12.

Problem context

Stack Overflow 2025 · AI section

Product and infrastructure

Vadhyar repository and build records

Graphify

CodeTour

OpenSpec

Vitest